

中山大學
SUN YAT-SEN UNIVERSITY

并行程序设计与算法实验

Lab8-并行多源最短路径搜索

姓 名 林宏宇

学 号 23320093

学 院 计算机学院

专 业 计算机科学与技术

2026 年 6 月 2 日

1 实验目的

- 评估并行化最短路径算法的性能。
- 探索不同最短路径算法的并行适应性。

2 实验内容

实现并行的多源最短路径搜索，具体要求如下：

- 使用 OpenMP、Pthreads 或 MPI 中的任意一种或多种并行编程模型来实现。

  你选择的并行框架是： OpenMP

- 选用任意一种最短路径算法作为基础进行并行化。

  你选用的算法是： 以每个顶点为源点独立执行 Dijkstra 算法（Dijkstra-per-Source），同时对 Floyd-Warshall 算法进行对比实验。

- 简要描述所选算法的实现要点。

  回答：

  1. **数据结构**：使用邻接表存储稀疏图，每条边存储目标节点、权重及下一条边索引。源点距离矩阵采用行主序连续内存布局以提升缓存性能。
  2. **Dijkstra 算法**：对每个源点独立运行堆优化的 Dijkstra 算法。使用二叉最小堆维护待松弛节点，时间复杂度为 $O((V+E)\log V)$ 每源点，全集为 $O(V(V+E)\log V)$。
  3. **并行策略**：外层源点循环并行化，使用 `parallel for` 结合动态调度（chunk size $= 1$），将不同源点分配到各线程独立运行。动态调度有效缓解了因不同源点计算量差异导致的负载不均问题。
  4. **Floyd-Warshall 对比实现**：中间节点 k 循环保持串行，内层源点 i 循环并行化（`parallel for` + `schedule(static)`），j 循环使用 SIMD 向量化。利用稀疏图特性进行提前剪枝：若 `dist`$[i][k]$ 为 INF 则跳过整个内层循环。

3 实验结果与分析

3.1 实验配置简述

- **所用算法：** Dijkstra-per-Source（主要）+ Floyd-Warshall（对比）
- **并行框架：** OpenMP

- **并行方式：**
  - Dijkstra-per-Source：外层源点循环并行化，动态调度（`schedule(dynamic, 1)`），每个线程独立运行完整 Dijkstra 算法处理分配给它的源点子集。
  - Floyd-Warshall：中间节点 k 循环保持串行，内层源点 i 循环并行化，静态调度（`schedule(static)`），j 循环使用 SIMD 向量化加速。
- **测试数据特征：**
  - 数据集 1（Mouse Brain）：节点数量 $V_1 = 525$，边数量 $E_1 = 14691$，平均度数 $D_1 \approx 55.97$，边权范围 $[1.00, 1.11]$，稀疏度 $\frac{2E}{V(V-1)} \approx 10.7\%$
  - 数据集 2（Flower Network）：节点数量 $V_2 = 930$，边数量 $E_2 = 13521$，平均度数 $D_2 \approx 29.08$，边权均为 1，稀疏度 $\frac{2E}{V(V-1)} \approx 3.1\%$
- **实验环境：** Ubuntu 22.04, GCC 11.4.0, OpenMP 4.5, 96 核 CPU（实验中限制使用 1–16 线程），每次配置重复运行 5 次取平均值。

3.2 数据集 1：Mouse Brain 网络性能数据

数据集 1 (节点数量: 525, 平均度数: 55.97) 的性能数据：

表 1: 数据集 1 (525 节点, 平均度数 55.97) 上 Dijkstra-per-Source 的并行性能 (串行时间 T_s: 0.1422 秒)

线程数量 (p)	**运行时间** T_p **(秒)**	**加速比** $S_p = T_s/T_p$	**并行效率** $E_p = S_p/p$
1	0.1422	1.00	1.00
2	0.0801	1.78	0.89
4	0.0488	2.92	0.73
8	0.0328	4.34	0.54
16	0.0228	6.24	0.39

表 2: 数据集 1 (525 节点, 平均度数 55.97) 上 Floyd-Warshall 的并行性能 (串行时间 T_s: 0.1205 秒)

线程数量 (p)	**运行时间** T_p **(秒)**	**加速比** $S_p = T_s/T_p$	**并行效率** $E_p = S_p/p$
1	0.1205	1.00	1.00
2	0.0792	1.52	0.76
4	0.0537	2.24	0.56
8	0.0390	3.09	0.39
16	0.0296	4.07	0.25

3.3 数据集 2：Flower Network 性能数据

数据集 2 (节点数量: 930, 平均度数: 29.08) 的性能数据：

表 3: 数据集 2 (930 节点, 平均度数 29.08) 上 Dijkstra-per-Source 的并行性能 (串行时间 T_s: 0.0813 秒)

线程数量 (p)	**运行时间** T_p **(秒)**	**加速比** $S_p = T_s/T_p$	**并行效率** $E_p = S_p/p$
1	0.0813	1.00	1.00
2	0.0503	1.62	0.81
4	0.0337	2.41	0.60
8	0.0232	3.51	0.44
16	0.0157	5.17	0.32

表 4: 数据集 2 (930 节点, 平均度数 29.08) 上 Floyd-Warshall 的并行性能 (串行时间 T_s: 0.4718 秒)

线程数量 (p)	**运行时间** T_p **(秒)**	**加速比** $S_p = T_s/T_p$	**并行效率** $E_p = S_p/p$
1	0.4718	1.00	1.00
2	0.3348	1.41	0.70
4	0.2000	2.36	0.59
8	0.1195	3.95	0.49
16	0.0788	5.98	0.37

3.4 并行性能分析

3.4.1 运行时间随线程数增加的变化趋势

回答：四种实验配置的运行时间均随线程数增加而单调递减，但递减速率逐渐放缓，呈现出典型的 Amdahl 定律特征。

- **Dijkstra (Mouse)**：从 1 线程的 0.1422s 降至 16 线程的 0.0228s，加速比为 6.24x，运行时间缩短至原来的 16.0%。
- **Dijkstra (Flower)**：从 1 线程的 0.0813s 降至 16 线程的 0.0157s，加速比为 5.17x，运行时间缩短至原来的 19.3%。注意，Flower 数据集的串行运行时间反而比 Mouse 更短（0.0813s vs 0.1422s），这是因为 Dijkstra 算法的时间复杂度依赖于边数和度数——Mouse 网络的平均度数（55.97）几乎是 Flower 网络（29.08）的两倍，每次松弛操作需要检查更多邻接边，导致单源 Dijkstra 的计算量显著更大。

- **Floyd-Warshall (Mouse)**：从 0.1205s 降至 0.0296s。
- **Floyd-Warshall (Flower)**：从 0.4718s 降至 0.0788s。Floyd-Warshall 的串行运行时间随节点数增长显著（$O(V^3)$：$525^3 \approx 1.45 \times 10^8$ vs $930^3 \approx 8.04 \times 10^8$），Flower 的操作量约为 Mouse 的 5.5 倍，实测时间比例约为 3.9 倍（0.4718/0.1205），差异由稀疏性带来的提前剪枝优化效果不同所致。

3.4.2 加速比的变化情况

回答：加速比随线程数增加而持续增长，但增长率逐渐降低。

- **Dijkstra (Mouse)** 加速比最高，16 线程达到 6.24x。原因是：(1) 各源点的 Dijkstra 任务完全独立，不存在迭代间同步开销；(2) 动态调度有效平衡了不同源点计算量的差异。
- **Dijkstra (Flower)** 加速比相对较低（16 线程 5.17x），主因是串行基线时间短（仅 0.0813s），线程管理开销占比更大。单个 Dijkstra 源点计算量更小（平均度数低），动态调度的任务粒度更细，开销比例上升。
- **Floyd-Warshall (Flower)** 在两种 FW 实现中获得更高加速比（5.98x vs Mouse 的 4.07x），因为更大的问题规模（$V = 930$，$V^3 \approx 8.04 \times 10^8$）更好地摊还了并行开销。但 FW 的绝对加速比仍低于 Dijkstra，原因在于 FW 每次 k 迭代都需要线程 fork/join，V 次同步开销累积显著。
- **Floyd-Warshall (Mouse)** 加速比最低（16 线程仅 4.07x），问题规模小（$V = 525$）且同步开销占比最高。

3.4.3 并行效率的变化情况及影响因素

回答：并行效率随线程数增加而下降，这是并行计算中的普遍现象。具体分析：

1. **效率下降趋势**：
   - Dijkstra (Mouse)：效率从 $p = 2$ 的 0.89 降至 $p = 16$ 的 0.39
   - Dijkstra (Flower)：从 0.81 降至 0.32
   - Floyd-Warshall (Mouse)：从 0.76 降至 0.25
   - Floyd-Warshall (Flower)：从 0.70 降至 0.37
2. **影响并行效率的主要因素**：

   (a) 同步开销（Synchronization Overhead）：Floyd-Warshall 算法的效率显著低于 Dijkstra，核心原因在于 FW 需要在每次 k 迭代末尾进行隐式 barrier 同步

（`#pragma omp parallel for` 包含隐式同步）。对于 $V = 930$ 的图，这意味着 930 次线程 fork/join 操作。每次 fork/join 涉及线程唤醒、工作分配和同步等待，累积开销随线程数增加而放大。

(b) 任务粒度（Task Granularity）：Dijkstra 算法以整个源点的最短路径树计算为任务单元，粒度较粗（每个任务 $O((V+E)\log V)$）；Floyd-Warshall 以单个源点行的一次 k 迭代为任务单元，粒度较细（$O(V)$）。当问题规模不足以摊还并行开销时，细粒度任务的效率损失更明显。

(c) 负载均衡（Load Balance）：Dijkstra 算法中不同源点的计算量可能存在差异——位于图中心的源点需要探索更多节点。本实验采用 `schedule(dynamic, 1)` 动态调度策略，每次分配一个源点任务，有效缓解了负载不均问题。

(d) 内存带宽竞争（Memory Bandwidth Contention）：两种算法都是内存密集型——Dijkstra 频繁访问邻接表和距离矩阵，Floyd-Warshall 执行大量矩阵读写。随着线程数增加，多核共享的最后一级缓存（LLC）和内存带宽成为瓶颈。在 16 线程下，多个核心同时访问主存导致带宽饱和，限制了进一步加速。

(e) 串行部分（Amdahl's Law）：图数据加载、结果矩阵初始化、测试查询处理等串行部分占总执行时间的比例虽小，但随着并行部分加速，串行部分占比相对增大，成为性能瓶颈。

3.4.4 不同数据特征对并行性能的影响

回答：

1. **节点数量（V）的影响**：

   - 对 Floyd-Warshall：节点数量是最关键因素。FW 时间复杂度为 $O(V^3)$，V 从 525 增至 930（增加 77%），计算量增加约 455%。Flower 数据集上 FW 的加速比（5.98x）显著优于 Mouse（4.07x），因为更大的计算量更好地摊还了 V 次同步开销。这验证了 Gustafson 定律：问题规模增大时，并行效率可以得到改善。
   - 对 Dijkstra：节点数量的影响通过平均度数间接体现。虽然 Flower 的 V 更大，但因其平均度数低（29 vs 56），总计算量 $O(V(V+E)\log V)$ 反而更小——Flower 每个源点平均遍历更少的边。这解释了 Flower 上 Dijkstra 的加速比（5.17x）反而不如 Mouse（6.24x）。

2. **平均度数（D）的影响**：

   - 对 Dijkstra：平均度数直接决定每个源点的 Dijkstra 计算量。度数越高，每次从堆中弹出节点时需要松弛的邻接边越多，单个任务的计算时间越长。Mouse

网络的高度数（55.97 vs 29.08）使得每个源点任务更重，任务粒度更粗，因此并行效率更高（$p = 16$ 时 $E = 0.39$ vs 0.32）。

- 对 Floyd-Warshall：平均度数影响提前剪枝优化的效果。在稀疏图中（如 Flower，稀疏度 3.1%），大量 `dist`$[i][k]$ 在迭代初期为无穷大，提前剪枝跳过了大量无用计算，实际计算量远小于 V^3 的理论峰值。随着 k 增大、可达路径增多，计算密度逐渐上升。

3. **算法-数据匹配性**：

实验结果表明，对于稀疏图（两个数据集稀疏度均在 3–11%），Dijkstra-per-Source 在绝对性能上优于 Floyd-Warshall：

- Mouse 数据集（$D \approx 56$）：Dijkstra 1 线程 0.1422s vs FW 1 线程 0.1205s（FW 略快），但 16 线程下 Dijkstra（0.0228s）明显快于 FW（0.0296s）
- Flower 数据集（$D \approx 29$）：Dijkstra 在所有线程数下均显著优于 FW（1 线程：0.0813s vs 0.4718s；16 线程：0.0157s vs 0.0788s）

图的稀疏性越高，Dijkstra 相对 Floyd-Warshall 的优势越大。如果图变得更稠密（如 $D > 100$），Floyd-Warshall 的 $O(V^3)$ 固定计算模式优势将逐渐显现。

3.5 算法对比总结

表 5: 两种并行算法在不同数据集上的 16 线程性能对比

算法	**数据集**	T_1 **(s)**	T_{16} **(s)**	S_{16}
Dijkstra-per-Source	Mouse (525, D=56)	0.1422	0.0228	6.24
Dijkstra-per-Source	Flower (930, D=29)	0.0813	0.0157	5.17
Floyd-Warshall	Mouse (525, D=56)	0.1205	0.0296	4.07
Floyd-Warshall	Flower (930, D=29)	0.4718	0.0788	5.98

结论：

- 对于稀疏图，Dijkstra-per-Source 结合动态调度是更优的并行策略，因为它消除了迭代间的同步开销，且任务粒度更粗。
- Floyd-Warshall 在节点数增大时展现出更好的可扩展性（Flower 上 $S_{16} = 5.98$），适合稠密图或中等规模图的全对最短路径批量计算。
- 并行框架 OpenMP 能够以较少的代码修改实现显著的性能提升，循环级并行化（`parallel for`）是最高效的并行模式之一。

- 实际应用中应根据图的规模和稀疏度选择合适的算法：稀疏图优先选择 Dijkstra-per-Source，稠密图或节点数适中的图可选择 Floyd-Warshall。

4 附录：测试查询输出示例

为验证算法正确性，对两个数据集各生成 20 组随机节点对作为测试查询，两种算法（Dijkstra-per-Source 和 Floyd-Warshall）在所有测试查询上输出完全一致，验证了实现的正确性。

数据集 1（Mouse）测试查询结果（前 5 组）：

- (17, 173): 5.305714
- (362, 221): 5.299188
- (108, 131): 8.684543
- (334, 453): 5.356772
- (158, 13): 6.511203

数据集 2（Flower）测试查询结果（前 5 组）：

- (428, 814): 9.000000
- (876, 621): 5.000000
- (414, 779): 7.000000
- (199, 632): 5.000000
- (200, 791): 7.000000